

Homework 2

Inventory Demand Forecasting with Returns

Grupo Bimbo Kaggle Competition

1 AI for Retail Homework

This homework tackles one of the most operationally complex problems in consumer goods: predicting *net* product demand at the point of sale when returns are a significant and structured part of the business. You are expected — and encouraged — to use advanced AI-assisted development techniques throughout. The goal is not to write every line of code by hand, but to *understand* every line that ends up in your submission.

1.1 How to Use Claude Code or Similar Assistants to Learn While Building

AI coding assistants like **Claude Code** can be a tremendous accelerator — but used carelessly, they can also short-circuit your learning entirely. The difference lies in *how* you interact with them. This section gives you a workflow that lets you move fast while still genuinely understanding what you build.

1.2 The Core Principle: Drive, Don't Passenger

Think of Claude Code as a very knowledgeable co-pilot who can type faster than you. The moment you hand over the wheel entirely — pasting a prompt, accepting the output, and moving on — you stop learning. The goal is to stay in the driver's seat: you decide *what* to build and *why*, and you use the assistant to help you figure out *how*.

△ Important Note

A useful rule of thumb: **never accept code you cannot read aloud and explain line by line.**

1.3 A Recommended Workflow

- 1. Attempt it yourself first.** Before opening Claude Code, spend 15–20 minutes trying to solve the problem on your own. Even if you get stuck, this primes your brain with the right questions and makes the assistant's answer far more meaningful.
- 2. Ask for explanation alongside code.** Always append “*and explain each part*” or “*walk me through the logic*” to your prompts. Reading an explanation forces you to check your understanding and spot gaps.
- 3. Ask “why”, not just “what”.** After receiving code, follow up with questions like “*Why did you use a log transformation here instead of raw units?*” or “*What would happen to my RMSLE if I predicted negative demand?*”. These follow-up questions are where the real learning happens.

4. **Break it deliberately.** Once you have working code, modify something — remove a feature, change the loss function, introduce a small data leak — and predict what will happen before running it. Then run it and see if you were right.
5. **Re-implement from memory.** After understanding a piece of code, close it and rewrite it yourself from scratch. The act of reconstructing it consolidates your understanding far better than re-reading it.
6. **Use Claude to review your own code.** Once you have written something yourself, ask Claude Code to critique it: *“What are the weaknesses of this feature engineering pipeline?”* or *“How would a Kaggle Grandmaster improve this baseline?”*. This gives you targeted feedback without replacing your creative effort.

1.4 Prompts That Teach vs. Prompts That Replace

The way you phrase a prompt determines whether you learn or just receive an answer.

Prompts that replace learning	Prompts that build understanding
“Write me the full forecasting pipeline.”	“I have written this lag-feature function — what is wrong with it and why?”
“Give me the code for XGBoost regression.”	“Explain why RMSLE is a better loss for this problem than RMSE, then show me the minimal code.”
“Fix my data leakage bug.”	“Here is my feature engineering code and my suspiciously perfect validation score. What data leakage pattern am I introducing?”
“Add geospatial features to my model.”	“What does it mean to geocode an agency, and how would latitude/longitude features help a tree-based model capture regional demand patterns?”

1.5 An Honest Note on Effort and Grades

It is entirely possible to complete this homework in under an hour using Claude Code with minimal understanding. It is also entirely possible to spend three hours with it and come away with a solid grasp of demand forecasting, feature engineering for tabular data, and applied geospatial analytics. **The tool does not determine the outcome — your intellectual engagement does.**

The reflection document you submit, and any follow-up questions during evaluation, are specifically designed to surface whether you understood what you built. A model that achieves a low RMSLE but that you cannot explain will not receive full marks. Use the assistant. Learn from it. Own what you submit.

2 Learning Objectives

In this individual homework you will build a demand forecasting system for Grupo Bimbo's distribution network, going through two progressive stages: a baseline model and an enhanced version with geospatial and temporal features.

By the end of this assignment you will be able to:

- Explain why inventory returns are structurally different from other forecasting signals, and why they make demand prediction harder.
- Distinguish between *gross* sales, *returns*, and *net* demand, and compute the correct target variable.
- Build a baseline forecasting model using lag features and gradient boosting.
- Engineer geospatial features from agency location data and interpret their influence on local demand patterns.
- Implement three targeted improvements: return-rate features, spatial clustering, and product-channel interaction features.
- Build a Streamlit interface that lets a user select a route and agency and inspect predicted demand versus historical actuals.
- Write a critical reflection that honestly evaluates your model's strengths and remaining failure modes.

3 Motivation: Why Returns Are So Hard in Retail and CPG

3.1 Returns Are Not Just Negative Sales

In most industries, a return is an exception: a customer changes their mind and sends a product back. In the *direct store delivery* (DSD) model used by bakery and snack companies like Grupo Bimbo, returns are a **structural part of the business**. At every weekly visit, a sales representative simultaneously delivers new product and collects unsold product from the previous week. The net transfer is:

$$\text{Net Demand}_t = \text{Gross Sales}_t - \text{Returns}_t = \text{Adjusted Demand}_t$$

This creates at least four compounding difficulties that do not exist in classical demand forecasting settings.

3.2 Difficulty 1 — The Simultaneity Problem

Sales and returns happen at the *same visit*, so a retailer's "demand" for any given week is not fully observed until the *next* visit. If you forecast too high and over-deliver, you will see high returns next week. If you forecast too low and under-deliver, you miss sales. The signal you are trying to predict is itself a function of your previous forecast errors. This feedback loop is absent in most e-commerce or subscription demand forecasting problems.

3.3 Difficulty 2 — Return Rates Are Product- and Channel-Specific

A croissant has a shelf life of two days; a packaged cookie may last three weeks. A convenience store near a university has very different consumption patterns from a supermarket in a residential neighbourhood. As a result, return rates are not a global constant you can subtract from gross sales. They are a function of:

- Product perishability (shelf life / Semana gap)
- Channel type (bodega, minisuper, supermercado)
- Geographic location (urban density, consumer income, weather)
- Day of week and season

Ignoring this structure and using a simple average return rate per SKU produces systematically biased forecasts.

3.4 Difficulty 3 — Zero-Inflation and Asymmetric Costs

A significant fraction of (product, agency) pairs have zero demand in a given week — either because the SKU was never stocked there or because all units were returned. Standard regression models will be pulled toward a non-zero prediction by the few high-demand outliers. Furthermore, the cost of *over-predicting* (stale product, disposal cost) is often different from the cost of *under-predicting* (stockout, lost sale, retailer dissatisfaction). This asymmetry calls for a loss function tailored to the business, not just RMSE.

3.5 Difficulty 4 — Geospatial and Temporal Confounding

Demand at a given agency is influenced by factors that are geospatial in nature but not represented in the raw transaction data. Routes close to schools generate weekly peaks during the school term. Agencies in coastal or tourist areas show strong seasonality. Urban versus rural agencies differ systematically in SKU mix and return behaviour. Without explicitly encoding location, a model trained on national data will average out these effects, producing predictions that are mediocre everywhere and excellent nowhere.

△ Important Note

The competition metric is **Root Mean Squared Logarithmic Error (RMSLE)**. Predictions must be non-negative. A prediction of -5 will raise an exception. *Always clip your predictions to zero before evaluating or submitting.*

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}$$

RMSLE penalises under-prediction more than over-prediction at low demand levels, which aligns with the business cost of stockouts.

3.6 Why Geolocalisation Matters

The Bimbo dataset does not include GPS coordinates directly. However, the `town` field in the agency table, combined with the Mexican state implied by the agency name prefix, allows us to geocode each agency and generate powerful spatial features. Geolocalisation enables:

- **Spatial clustering:** grouping agencies into regions with similar demand profiles (arid north, rainy south, urban core, tourist coast).
- **Distance to distribution centre:** a proxy for delivery cost and freshness; agencies farther from a depot may receive slightly older product, driving higher returns.
- **Urban density:** agencies in dense urban areas tend to turn over perishables faster, resulting in lower return rates.
- **Regional economic indicators:** linking agency locations to census data (income, population density) can improve predictions for price-sensitive SKUs.

4 The Dataset

4.1 Overview

You will use the **Grupo Bimbo Inventory Demand** dataset, publicly available on Kaggle. It was released by Grupo Bimbo — one of the world's largest baking companies, headquartered in Mexico — and spans 9 weeks of weekly sales data across thousands of distribution routes and retail agencies.



Figure 1: Bimbo Mexico Logo

4.2 Training Data (train.csv.zip)

The main transactional file. Each row represents one product delivered (and potentially partially returned) on a given route, agency, client, and week combination. This is the file from which all features and the training target are derived. The training set covers **weeks 3–9** and contains approximately 74 million rows.

Column	Type	Description
Semana	int8	Week number of the transaction. Training data covers weeks 3–9 ; test data covers weeks 10–11 .
Agencia_ID	int32	Unique identifier for the distribution <i>agency</i> (depot). Each agency serves a geographic sub-region of Mexico and dispatches multiple delivery routes. Joins to <code>town_state.csv</code> for location data.
Canal_ID	int8	Sales <i>channel</i> code. Key values: 1 = traditional trade (tienda / bodega), 2 = convenience store, 7 = supermarket. Return rates vary substantially by channel; this is one of the most important categorical features.
Ruta_SAK	int32	Delivery <i>route</i> identifier. Each route belongs to one agency and visits a fixed set of clients on a weekly schedule.
Cliente_ID	int32	Unique identifier for the <i>client</i> (retail point of sale). Joins to <code>cliente_tabla.csv</code> for the human-readable name.
Producto_ID	int32	Unique identifier for the <i>product</i> (SKU). Joins to <code>producto_tabla.csv</code> for the product name, which encodes weight, unit count, and brand.
Venta_uni_hoy	int32	Gross units delivered today. Units physically placed on the shelf at this visit. Always ≥ 0 .
Venta_hoy	float32	Gross sales value in Mexican pesos for units delivered today. Computed as unit price $\times$ Venta_uni_hoy. <i>Not available in the test set.</i>
Dev_uni_proxima	int32	Units returned at the next visit. Unsold units collected from the shelf one week later. This is the structural return signal and is recorded one week in arrears. <i>Not available in the test set.</i>
Dev_proxima	float32	Value in pesos of units returned at the next visit. <i>Not available in the test set.</i>
Demanda_uni_equil	int32	Target variable. Bimbo's adjusted net demand: $\max(0, \text{Venta_uni_hoy} - \text{Dev_uni_proxima})$. This is what the model must predict for weeks 10–11. Approximately 40% of rows have a value of zero.

△ Important Note

Returns timing. `Dev_uni_proxima` records returns that will be *collected next week*, not this week. The return value in week t was caused by the delivery in week $t - 1$. When building lag features, **never** use `Dev_uni_proxima` from the current week as an input feature — it is a component of the target and would introduce data leakage.

4.3 Test Data (`test.csv.zip`)

Contains the (week, agency, channel, route, client, product) combinations for which a demand prediction must be submitted. The sales and returns columns are absent by design.

Column	Type	Description
id	int32	Row identifier used to match predictions in the submission file. Must appear in <code>sample_submission.csv</code> exactly as-is.
Semana	int8	Week number. Test set covers weeks 10 and 11 only.
Agencia_ID	int32	Same encoding as in <code>train.csv</code> . All test agencies also appear in the training set.
Canal_ID	int8	Same encoding as in <code>train.csv</code> .
Ruta_SAK	int32	Same encoding as in <code>train.csv</code> .
Cliente_ID	int32	Same encoding as in <code>train.csv</code> . A small number of client IDs in the test set do not appear in training (<i>cold-start</i> clients); handle these with a fallback prediction such as the route-level or agency-level median demand.
Producto_ID	int32	Same encoding as in <code>train.csv</code> . Similarly, a small number of products may be cold-start in the test set.

4.4 Product Catalogue (`producto_tabla.csv.zip`)

Maps each `Producto_ID` to a human-readable product name. The name string is rich: it encodes the brand, flavour, **weight in grams** (e.g. “680g”), and **units per pack** (e.g. “6Pz”). Parsing these attributes with regular expressions produces powerful numeric features.

Column	Type	Description
Producto_ID	int32	Unique product identifier. Primary key; joins to <code>train.csv</code> and <code>test.csv</code> .
NombreProducto	string	Full product name in Spanish. Contains embedded attributes: weight (e.g. “680g”, “1.2Kg”), units per pack (e.g. “6Pz”, “10Pz”), and occasionally format indicators (“Caja”, “Bolsa”). There are approximately 1,800 unique products. Parse with regular expressions to extract numeric features (see Part C, Improvement 3).

✓ Hint

Use these regular expressions to extract structured attributes from `NombreProducto`:

```
import re, numpy as np

def extract_weight_g(name):
    m = re.search(r"(\d+(?:\.\d+)?)\s*[Gg]", name)
    return float(m.group(1)) if m else np.nan

def extract_units(name):
    m = re.search(r"(\d+)\s*[Pp]z", name)
    return float(m.group(1)) if m else 1.0
```

Products with a high `weight_per_unit` (heavy single-unit items such as fresh bread loaves) are typically more perishable than small individually-wrapped items and therefore show structurally higher return rates.

4.5 Client Table (`cliente_tabla.csv.zip`)

Maps each `Cliente_ID` to the name of the retail point of sale. Client names often reveal the channel type (e.g. “OXXO” = convenience chain; “Walmart” = modern trade), providing a complementary channel signal alongside `Canal_ID`.

Column	Type	Description
Cliente_ID	int32	Unique client (retailer) identifier. Primary key; joins to <code>train.csv</code> and <code>test.csv</code> .
NombreCliente	string	Name of the retail client in Spanish. Can be a chain store name (“OXXO”, “7-ELEVEN”, “Walmart”), a personal name (small independent shops), or the placeholder “NO IDENTIFICADO”. Approximately 880,000 unique values; the majority are independent small retailers with sparse transaction histories — making raw <code>Cliente_ID</code> a high-cardinality feature that requires careful handling.

✓ Hint

Because most clients are small independent stores with very few weeks of history, avoid using raw `Cliente_ID` directly as a model feature. Instead, extract chain membership from `NombreCliente` using keyword matching (e.g. "OXXO" in `name`), or use target-encoded client-level median demand computed strictly on the training split.

4.6 Agency Geography (`town_state.csv.zip`)

The key file for geolocalisation. Provides the town and state for each distribution agency, enabling geocoding to latitude/longitude and the generation of spatial features (see Section 5).

Column	Type	Description
Agencia_ID	int32	Unique agency (distribution depot) identifier. Primary key; joins to <code>train.csv</code> and <code>test.csv</code> .
Town	string	Name of the Mexican town or city where the agency is located. Formatted as an internal code followed by the place name, e.g. "2058 AG. AZCAPOTZALCO". Strip the numeric prefix and any abbreviation tags ("AG.", "SUC.", "BODEGA") before geocoding.
State	string	Mexican state name in Spanish (e.g. "CIUDAD DE MEXICO", "JALISCO", "NUEVO LEON"). Append to the cleaned town name when geocoding to resolve ambiguous place names.

△ Important Note

The `Town` field contains internal agency codes prepended to the place name. You **must** strip the numeric prefix and abbreviation tags before passing the string to a geocoder — otherwise most queries will fail. Use:

```
import re
clean_town = re.sub(r"^\d+\s*(AG\.|SUC\.|BODEGA\s*)?", "", raw_town).strip()
query = f"{clean_town}, {state}, Mexico"
```

4.7 Submission Format (`sample_submission.csv.zip`)

Defines the exact format required for a Kaggle submission. Every `id` in this file must receive a corresponding non-negative prediction.

Column	Type	Description
id	int32	Row identifier matching the id column in test.csv. Rows must appear in the same order.
Demanda_uni_equil	float32	Your predicted net demand for this (week, agency, channel, route, client, product) combination. Must be ≥ 0 . The competition evaluates predictions with RMSLE; always clip before writing the file.

Listing 1: Generating a valid Kaggle submission file

```
import pandas as pd
import numpy as np

sample_sub = pd.read_csv("data/sample_submission.csv")

# test_df must have columns 'id' and 'pred'
submission = sample_sub[["id"]].merge(
    test_df[["id", "pred"]], on="id", how="left"
)
submission["Demanda_uni_equil"] = np.clip(
    submission["pred"].fillna(0), 0, None
)
submission[["id", "Demanda_uni_equil"]].to_csv(
    "outputs/submission.csv", index=False
)
print(submission["Demanda_uni_equil"].describe())
```

4.8 Key Column Definitions

- **Semana:** Week number (3–9 in training, 10–11 in test).
- **Agencia_ID:** Distribution agency (depot) identifier. Each agency serves a geographic area.
- **Canal_ID:** Sales channel (1 = traditional trade / bodega, 2 = convenience, 7 = supermarket, etc.).
- **Ruta_SAK:** Delivery route identifier. Each route belongs to one agency and visits a set of clients.
- **Producto_ID:** Product SKU. Product names in producto_tabla.csv often encode weight (e.g. “680g”) and units per pack.
- **Demanda_uni_equil:** The **target variable**. Computed by Bimbo as net adjusted demand. It is always ≥ 0 .

4.9 Scale and Statistics

- ~74 million rows in the training set.
- ~552 unique agencies across Mexico.
- ~1,800 unique products.
- ~880,000 unique clients.
- Return rates vary from 0% to over 60% depending on product and channel.

- Approximately 40% of rows have `Demanda_uni_equil = 0`.

✓ Hint

The full `train.csv` is ~800 MB. For development, sample 10–20 weeks of data from a subset of agencies. Always validate that your sampling does not break the temporal order needed for lag features.

```
import pandas as pd

# Load only a sample of agencies for fast iteration
df = pd.read_csv("data/train.csv", dtype={"Semana": "int8"})
sample_agencies = df["Agencia_ID"].drop_duplicates().sample(50, random_state=42)
df_small = df[df["Agencia_ID"].isin(sample_agencies)]
```

4.10 Loading and Preparing the Data

Listing 2: Loading and joining all Bimbo tables

```
import pandas as pd
import numpy as np

train = pd.read_csv("data/train.csv")
test = pd.read_csv("data/test.csv")
products = pd.read_csv("data/producto_tabla.csv")
clients = pd.read_csv("data/cliente_tabla.csv")
towns = pd.read_csv("data/town_state.csv")

# Merge agency geographies
train = train.merge(towns, on="Agencia_ID", how="left")
test = test.merge(towns, on="Agencia_ID", how="left")
train = train.merge(products, on="Producto_ID", how="left")
test = test.merge(products, on="Producto_ID", how="left")

# Compute return rate (available only in train)
train["return_rate"] = (
    train["Dev_uni_proxima"] / (train["Venta_uni_hoy"] + 1)
)

print(train[["Demanda_uni_equil", "return_rate"]].describe())
```

5 Geolocalisation: Adding Spatial Intelligence

5.1 Why Geocode the Agencies?

The `town_state.csv` file gives us a free-text town name and a Mexican state for each agency. By converting these to latitude/longitude coordinates we unlock a new family of features that tree-based

models can exploit.

5.2 Geocoding with GeoPy

We use geopy with the Nominatim geocoder (OpenStreetMap) to look up coordinates. Because Nominatim has a rate limit of one request per second, we cache results and geocode once.

Listing 3: Geocoding agencies from town and state names

```
from geopy.geocoders import Nominatim
from geopy.extra.rate_limiter import RateLimiter
import pandas as pd
import time

towns = pd.read_csv("data/town_state.csv")

geolocator = Nominatim(user_agent="bimbo_hw4")
geocode = RateLimiter(geolocator.geocode, min_delay_seconds=1)

def geocode_town(row):
    query = f"{row['Town']}, {row['State']}, Mexico"
    try:
        loc = geocode(query)
        if loc:
            return loc.latitude, loc.longitude
    except Exception:
        pass
    return None, None

towns[["lat", "lon"]] = towns.apply(
    geocode_town, axis=1, result_type="expand"
)
towns.to_csv("data/town_state_geocoded.csv", index=False)
print(f"Geocoded {towns['lat'].notna().sum()} / {len(towns)} agencies")
```

△ Important Note

Nominatim is a free public service. **Do not run more than one request per second.** The RateLimiter wrapper handles this automatically. Some towns will fail to geocode (ambiguous names or small villages); fill missing coordinates with the state centroid as a fallback.

5.3 Generating Spatial Features

Once coordinates are available, we can generate a rich set of spatial features.

Listing 4: Computing spatial features from geocoded coordinates

```

import numpy as np
from sklearn.cluster import KMeans

towns = pd.read_csv("data/town_state_geocoded.csv")
towns = towns.dropna(subset=["lat", "lon"])

# --- 1. Spatial clusters (regions) ---
km = KMeans(n_clusters=8, random_state=42, n_init="auto")
towns["geo_cluster"] = km.fit_predict(towns[["lat", "lon"]])

# --- 2. Distance to national distribution hub (Mexico City approx.) ---
MEX_LAT, MEX_LON = 19.43, -99.13

def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius in km
    phi1, phi2 = np.radians(lat1), np.radians(lat2)
    dphi = np.radians(lat2 - lat1)
    dlam = np.radians(lon2 - lon1)
    a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2)*np.sin(dlam/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))

towns["dist_to_hub_km"] = haversine(
    towns["lat"], towns["lon"], MEX_LAT, MEX_LON
)

# --- 3. Latitude zone (north / central / south) ---
towns["lat_zone"] = pd.cut(
    towns["lat"],
    bins=[0, 20, 23, 90],
    labels=["south", "central", "north"]
)

print(towns[["Agencia_ID", "geo_cluster", "dist_to_hub_km", "lat_zone"]].head())

```

✓ Hint

Try 6–12 clusters and inspect which agencies fall into each group. You should see geographically coherent regions emerge: the Baja California peninsula, the northern arid states (Sonora, Chihuahua), the central Bajio region, the Gulf Coast, and the southern states. Label these clusters meaningfully in your Streamlit app.

5.4 Visualising the Agency Network

Listing 5: Plotting the agency network with Folium

```

import folium
import pandas as pd

```

```
towns = pd.read_csv("data/town_state_geocoded.csv")

m = folium.Map(location=[23.6, -102.5], zoom_start=5)

cluster_colours = {
    0: "red", 1: "blue", 2: "green", 3: "purple",
    4: "orange", 5: "darkred", 6: "cadetblue", 7: "darkgreen"
}

for _, row in towns.dropna(subset=["lat", "lon"]).iterrows():
    colour = cluster_colours.get(int(row.get("geo_cluster", 0)), "gray")
    folium.CircleMarker(
        location=[row["lat"], row["lon"]],
        radius=5,
        color=colour,
        fill=True,
        popup=f"Agency {row['Agencia_ID']}: {row['Town']}, {row['State']}"
    ).add_to(m)

m.save("outputs/agency_map.html")
print("Map saved to outputs/agency_map.html")
```

6 What to Do in the Assignment

The assignment is composed of three parts. **Part A** is the baseline forecaster. **Part B** is the interactive Streamlit visualiser. **Part C** contains three possible improvements which are optional

6.1 Part A — Baseline Demand Forecaster

Build a gradient-boosted tree model (XGBoost or LightGBM) that predicts `Demanda_uni_equil` using only simple lag and aggregation features. Evaluate on weeks 8–9 (held out from training). Report your RMSLE. I provide you with an example that can give you an RMSE of around 0.49 in the Kaggle competition, which is quite good. If you want you can try to 'squeeze' the program to obtain a better result, but warning, is a very tricky competition where you can invest a ton of hours to improve only marginally the result.

6.1.1 Feature Engineering for the Baseline

The most powerful features for weekly demand forecasting are *lag features* — the demand observed one or more weeks ago for the same (product, agency, client) combination. Because we only have weeks 3–9, the maximum usable lag without data loss on the test set (week 10) is 3 weeks.

Listing 6: Baseline feature engineering with lag features

```

import pandas as pd
import numpy as np

def build_lag_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    Compute lag-1, lag-2, lag-3 demand per (Producto_ID, Agencia_ID, Cliente_ID).
    df must be sorted by Semana ascending before calling this function.
    """
    key = ["Producto_ID", "Agencia_ID", "Cliente_ID"]
    df = df.sort_values(["Semana"] + key).copy()

    for lag in [1, 2, 3]:
        df[f"demand_lag{lag}"] = (
            df.groupby(key) ["Demanda_uni_equil"].shift(lag)
        )

    # Rolling mean over the last 2 available weeks
    df["demand_roll2"] = (
        df.groupby(key) ["Demanda_uni_equil"]
        .transform(lambda x: x.shift(1).rolling(2, min_periods=1).mean())
    )
    return df

train = build_lag_features(train)

# Drop rows where lag features are NaN (first few weeks per group)
train_model = train.dropna(subset=["demand_lag1", "demand_lag2"])

FEATURES = [
    "Semana", "Agencia_ID", "Canal_ID", "Ruta_SAK",
    "Cliente_ID", "Producto_ID",
    "demand_lag1", "demand_lag2", "demand_lag3", "demand_roll2",
]
TARGET = "Demanda_uni_equil"

```

6.1.2 Training the Baseline Model

Listing 7: Training XGBoost with RMSLE-equivalent objective

```

import xgboost as xgb
from sklearn.metrics import mean_squared_log_error
import numpy as np

# Temporal train/validation split -- NEVER shuffle for time-series data
train_data = train_model[train_model["Semana"] <= 7]
val_data = train_model[train_model["Semana"] >= 8]

```

```

X_train = train_data[FEATURES]
y_train = np.log1p(train_data[TARGET]) # log1p transform as proxy for RMSLE
X_val = val_data[FEATURES]
y_val = val_data[TARGET]

model = xgb.XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    eval_metric="rmse",
    random_state=42,
    n_jobs=-1,
)

model.fit(
    X_train, y_train,
    eval_set=[(X_val, np.log1p(y_val))],
    verbose=100,
)

preds = np.expm1(model.predict(X_val)).clip(0) # inverse transform + clip
rmsle = np.sqrt(mean_squared_log_error(y_val, preds))
print(f"Validation RMSLE: {rmsle:.4f}")

```

△ Important Note

Never use a random train/test split for time-series data. If you shuffle the rows and train on week 9 to predict week 7, you are using future information to predict the past — a data leakage bug that inflates your validation score but destroys real-world performance. Always split on *Semana*.

6.1.3 Sanity Checks for the Baseline

After training, verify the model's behaviour qualitatively:

Listing 8: Sanity-checking the baseline predictions

```

import pandas as pd
import matplotlib.pyplot as plt

# Pick one agency + product with non-trivial demand history
sample = val_data[
    (val_data["Agencia_ID"] == 1114) &
    (val_data["Producto_ID"] == 1212)
]

```

```
].copy()
sample["pred"] = np.exp1(model.predict(sample[FEATURES])).clip(0)

print(sample[["Semana", TARGET, "pred", "demand_lag1"]].to_string())

# Feature importance
feat_imp = pd.Series(model.feature_importances_, index=FEATURES)
print(feat_imp.sort_values(ascending=False))
```

✓ Hint

You should see `demand_lag1` as the single most important feature by a large margin. If you see `Semana` or `Agencia_ID` dominating, your lag features may not have been computed correctly.

6.1.4 Leakage issues with feature engineering

This problem is very tricky, and there are many issues that make difficult to obtain a good score, one of the issues is Leakage. Leakage can occur due to the lag impacting feature engineering. This occurs if your Lag Features were calculated using a global average or a window that "looked" at Week 9.

- **Scenario A (Safe):** To calculate the `tminus1` feature for Week 9, you only use data from Week 8. This is how the model will have to work in the real world.
- **Scenario B (Leaked):** If, during feature engineering, you calculated a feature *AverageSalesPerClient* using the entire dataset (Weeks 3 to 9) and then joined that back to your training rows, your Week 3–8 rows now contain a "hint" about what happened in Week 9.

6.2 Part B — Streamlit Application

Build a Streamlit application (`app.py`) that allows an analyst to interactively explore demand predictions and historical patterns. The application must include:

- A **sidebar** with filters: Agency (dropdown), Product (dropdown), Canal (dropdown).
- A **demand history chart** showing actual demand per week for the selected (agency, product) pair, with the model's predictions overlaid.
- A **return rate panel** showing historical return rate (`Dev_uni_proxima / Venta_uni_hoy`) over time for the selected combination.
- A **map tab** displaying all agencies colour-coded by their spatial cluster, with the selected agency highlighted.
- A **prediction panel** showing the model's forecast for week 10, with a confidence interval derived from the training residuals.

Listing 9: Minimal Streamlit skeleton — extend this substantially

```
import streamlit as st
import pandas as pd
import numpy as np
```

```

import folium
from streamlit_folium import st_folium
import xgboost as xgb

@st.cache_resource
def load_model():
    import pickle
    with open("models/bimbo_baseline.pkl", "rb") as f:
        return pickle.load(f)

@st.cache_data
def load_data():
    train = pd.read_csv("data/train_features.csv") # pre-built with lag features
    towns = pd.read_csv("data/town_state_geocoded.csv")
    prods = pd.read_csv("data/producto_tabla.csv")
    return train, towns, prods

model, (train, towns, prods) = load_model(), load_data()

st.set_page_config(page_title="Bimbo Demand Forecaster", layout="wide")
st.title("Grupo Bimbo --- Demand Forecasting Dashboard")
st.markdown("----")

# - Sidebar filters -
agency_list = sorted(train["Agencia_ID"].unique())
product_list = sorted(prods["Producto_ID"].unique())

agency_id = st.sidebar.selectbox("Agency", agency_list)
product_id = st.sidebar.selectbox("Product", product_list)
canal_id = st.sidebar.selectbox("Channel", sorted(train["Canal_ID"].unique()))

# - Main tabs -
tab_hist, tab_map, tab_pred = st.tabs(
    ["Demand History", "Agency Map", "Week-10 Forecast"]
)

with tab_hist:
    mask = (
        (train["Agencia_ID"] == agency_id) &
        (train["Producto_ID"] == product_id) &
        (train["Canal_ID"] == canal_id)
    )
    hist = train[mask].groupby("Semana")[
        ["Demanda_uni_equil", "return_rate"]
    ].mean().reset_index()

    st.subheader("Actual Demand vs. Return Rate")
    st.line_chart(hist.set_index("Semana"))

```

```

with tab_map:
    st.subheader("Agency Network (colour = spatial cluster)")
    # TODO: render folium map with st_folium
    st.info("Build the map using the geocoded towns CSV and folium.")

with tab_pred:
    st.subheader("Forecast for Week 10")
    # TODO: build feature row for week 10 and call model.predict
    st.info("Connect your model's week-10 prediction here.")

```

△ Important Note

Use `@st.cache_data` for all dataframes and `@st.cache_resource` for the trained model. Pre-compute the lag features and save them to `data/train_features.csv` in a separate script so that the Streamlit app does not have to re-run the full feature engineering pipeline on every launch.

6.3 Part C — Three Improvements Possible improvements

You must implement **all three** of the following improvements on top of the baseline to achieve full marks on this section.

6.3.1 Improvement 1 — Return-Rate Features

Motivation. The baseline ignores the return process entirely, using only net demand lags. But historical return rates for a (product, channel, agency) combination are one of the strongest predictors of future returns — and therefore future net demand. If a product consistently has a 30% return rate at a given agency, the model should account for this structural pattern rather than being surprised every week.

Implementation. For each (product, agency, channel) triplet, compute a rolling historical return rate using only data prior to the current week (to avoid data leakage):

$$\hat{r}_t(p, a, c) = \frac{\sum_{s < t} \text{Dev}_s}{\sum_{s < t} \text{Venta}_s + 1}$$

Add this as a feature alongside the demand lags.

Listing 10: Computing rolling historical return rates

```

import pandas as pd
import numpy as np

def add_return_rate_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    Computes a leakage-free historical return rate for each
    (Producto_ID, Agencia_ID, Canal_ID) triplet.

```

```

"""
key = ["Producto_ID", "Agencia_ID", "Canal_ID"]
df = df.sort_values(["Semana"] + key).copy()

# Cumulative sums shifted by 1 week to avoid leakage
df["cum_dev"] = df.groupby(key)["Dev_uni_proxima"].transform(
    lambda x: x.shift(1).expanding().sum()
)
df["cum_venta"] = df.groupby(key)["Venta_uni_hoy"].transform(
    lambda x: x.shift(1).expanding().sum()
)
df["hist_return_rate"] = df["cum_dev"] / (df["cum_venta"] + 1)

# Also add channel-level global return rate as a fallback
global_rr = (
    df.groupby(["Canal_ID", "Semana"])
    .apply(lambda g: g["Dev_uni_proxima"].sum() /
            (g["Venta_uni_hoy"].sum() + 1))
    .reset_index(name="canal_return_rate")
)
df = df.merge(global_rr, on=["Canal_ID", "Semana"], how="left")

return df

```

✓ Hint

After adding return-rate features, inspect the SHAP values of your model. You should see `hist_return_rate` rise significantly in importance, especially for perishable SKUs. If it does not, check whether you accidentally included the current week's returns in the cumulative sum (data leakage).

6.3.2 Improvement 2 — Spatial Cluster Features

Motivation. The baseline treats each `Agencia_ID` as an opaque integer. A tree model can memorise the training behaviour of each agency but cannot generalise to agencies it has seen less frequently. By replacing the raw agency ID with a geographically coherent *cluster label*, we allow the model to share information across agencies with similar demand environments.

Implementation. Use the geocoded coordinates from Section 5 to assign each agency to one of K spatial clusters. Then:

1. Replace (or supplement) `Agencia_ID` with `geo_cluster`.
2. Add `dist_to_hub_km` and `lat_zone` as numeric features.
3. Compute cluster-level median return rates as an additional feature.

Listing 11: Integrating spatial features into the model

```

import pandas as pd
from sklearn.cluster import KMeans
import numpy as np

towns = pd.read_csv("data/town_state_geocoded.csv").dropna(subset=["lat", "lon"])

km = KMeans(n_clusters=8, random_state=42, n_init="auto")
towns["geo_cluster"] = km.fit_predict(towns[["lat", "lon"]])

# Haversine distance to Mexico City
towns["dist_to_hub_km"] = haversine(
    towns["lat"], towns["lon"], 19.43, -99.13
)

# Merge spatial features into train/test
train = train.merge(
    towns[["Agencia_ID", "geo_cluster", "dist_to_hub_km"]],
    on="Agencia_ID", how="left"
)
test = test.merge(
    towns[["Agencia_ID", "geo_cluster", "dist_to_hub_km"]],
    on="Agencia_ID", how="left"
)

# Cluster-level median demand (computed from training data only)
cluster_stats = (
    train.groupby("geo_cluster")["Demanda_uni_equil"]
    .median()
    .reset_index(name="cluster_median_demand")
)
train = train.merge(cluster_stats, on="geo_cluster", how="left")
test = test.merge(cluster_stats, on="geo_cluster", how="left")

FEATURES_SPATIAL = FEATURES + [
    "geo_cluster", "dist_to_hub_km", "cluster_median_demand"
]

```

△ Important Note

Cluster statistics must be computed from the *training split only* and then applied to the validation set. Computing them from the full dataset before splitting constitutes target leakage and will make your validation score unreliable.

6.3.3 Improvement 3 — Product-Channel Interaction Features

Motivation. The demand for a premium pastry in a supermarket is structurally different from the demand for the same product in a small convenience store. The model needs explicit features that capture

the *interaction* between product type and sales channel, rather than learning it purely from the raw ID combination (which may be sparse for rare product-channel pairs).

Implementation. Parse useful attributes from the product name (weight, unit count, freshness proxy) and cross them with the channel ID to create interpretable interaction features. Then add median demand aggregates at the (product family $\times$ channel) level.

Listing 12: Product attribute extraction and channel interaction features

```
import re
import pandas as pd

products = pd.read_csv("data/producto_tabla.csv")

def extract_weight_g(name: str) -> float:
    """Extract weight in grams from product name string."""
    m = re.search(r"(\d+(?:\.\d+)?)\s*[Gg]", name)
    if m:
        return float(m.group(1))
    return np.nan

def extract_units(name: str) -> float:
    """Extract number of units per pack from product name."""
    m = re.search(r"(\d+)\s*[Pp]z", name)
    if m:
        return float(m.group(1))
    return 1.0

products["weight_g"] = products["NombreProducto"].apply(extract_weight_g)
products["units"] = products["NombreProducto"].apply(extract_units)

# Weight per unit as a freshness proxy: heavy single-unit items
# (e.g. 680g loaves) are more perishable than small multi-unit packs
products["weight_per_unit"] = products["weight_g"] / products["units"]

# Merge product attributes
train = train.merge(
    products[["Producto_ID", "weight_g", "units", "weight_per_unit"]],
    on="Producto_ID", how="left"
)
test = test.merge(
    products[["Producto_ID", "weight_g", "units", "weight_per_unit"]],
    on="Producto_ID", how="left"
)

# Product-channel median demand (train only)
pc_stats = (
    train.groupby(["Producto_ID", "Canal_ID"])["Demanda_uni_equil"]
    .agg(["median", "std"])
```

```

.reset_index()
.rename(columns={"median": "pc_median_demand",
                "std": "pc_std_demand"})
)
train = train.merge(pc_stats, on=["Producto_ID", "Canal_ID"], how="left")
test = test.merge(pc_stats, on=["Producto_ID", "Canal_ID"], how="left")

FEATURES_FULL = FEATURES_SPATIAL + [
    "weight_g", "units", "weight_per_unit",
    "pc_median_demand", "pc_std_demand",
    "hist_return_rate", "canal_return_rate",
]

```

✓ Hint

After adding all three improvements, run a SHAP analysis on your full model to understand which features are driving predictions. You should see: `demand_lag1` still dominant, followed by `hist_return_rate`, `pc_median_demand`, and `dist_to_hub.km`. Geospatial cluster membership often appears in the top 10 features for agencies in peripheral or tourist regions.

7 Deliverables and Submission

You must submit a single **zip file** named `HW4_YourLastName.zip` containing:

1. `notebooks/01_eda.ipynb` — Exploratory Data Analysis. Must include: distribution of `Demanda_uni_equil`, return rate by channel and product category, and a discussion of zero-inflation.
2. `notebooks/02_geolocation.ipynb` — Geocoding notebook. Must include: geocoding code, a Folium map of all agencies, and a discussion of what spatial patterns you observe.
3. `notebooks/03_baseline.ipynb` — Baseline model training and evaluation. Report validation RMSLE and discuss at least two failure modes visible in the residuals.
4. `notebooks/04_improvements.ipynb` — Side-by-side comparison of baseline vs. all three improvements.
5. `app.py` — The complete Streamlit application with demand history, map, and forecast tabs.
6. `reflection.pdf` (1–2 pages) covering:
 - What business question does your model answer?
 - Where does it fail, and what additional data would help?
 - How does the geospatial dimension change the forecasting problem?
 - One thing you genuinely learned from building this project.

 Important Note

Do not include raw dataset files in your zip — they are too large. Include a README.md with instructions for downloading the data from Kaggle (<https://www.kaggle.com/c/grupo-bimbo-inventory-demand/data>) and placing it in the data/ directory. Include any pre-processed files you generate (geocoded towns, feature tables) only if they are under 50 MB.

References

- [1] Grupo Bimbo. *Grupo Bimbo Inventory Demand*. Kaggle Competition Dataset, 2016. <https://www.kaggle.com/c/grupo-bimbo-inventory-demand/data>
- [2] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [4] S. Lundberg and S.-I. Lee. *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS), 2017. <https://github.com/slundberg/shap>
- [5] GeoPy Team. *GeoPy: Geocoding library for Python*. <https://geopy.readthedocs.io>, 2024.
- [6] python-visualization. *Folium: Python Data, Leaflet.js Maps*. <https://python-visualization.github.io/folium/>, 2024.
- [7] Streamlit Team. *Streamlit Documentation*. <https://docs.streamlit.io>, 2024.